

Introducere în Matlab

Laboratorul 1, DEDP

Table of contents

1	Obiectiv	1
2	Noțiuni teoretice	2
2.1	Mediul Matlab	2
2.2	Matematică de bază: Command Window ca un calculator	2
2.3	Variabile	3
2.4	Funcții și cuvinte cheie	4
2.5	Vectori și matrici	6
	Crearea vectorilor și matricilor	6
	Operatorul două puncte — șiruri regulat spațiate	7
	linspace — un număr dat de puncte între două valori	7
	Dimensiuni și mărimi	8
2.6	Operații pe vectori și matrici	8
2.7	Indexare: accesarea elementelor unui vector	9
	Indexare logică	9
2.8	Grafică de bază	10
2.9	Rezumat: funcții și notații pe care trebuie să le cunoașteți	10
3	Exerciții	12
4	Întrebări finale	14

1 Obiectiv

Familiarizarea cu mediul Matlab: Command Window (Fereastra de Comenzi), Workspace și Editor. Învățarea sintaxei de bază Matlab: operatori aritmetici, variabile, crearea și manipularea vectorilor și matricilor.

Stilul de lucru pe tot parcursul semestrului: introduceți fiecare comandă și citiți rezultatul. Dacă rezultatul nu este cel așteptat, acesta este un moment de învățare, nu o eroare.

2 Noțiuni teoretice

2.1 Mediul Matlab

La pornirea Matlab, fereastra desktop conține trei părți pe care le vom folosi tot semestrul:

- **Command Window** (Fereastra de Comenzi) — panoul central în care tastați expresii și vedeți imediat rezultatul. Fiecare comandă se tastează după promptul `>>`.
- **Workspace** (Zona de lucru) — panoul lateral care listează toate variabilele existente în memorie, împreună cu dimensiunea și valoarea lor.
- **Editor** — o fereastră separată (butonul „New Script” sau `edit` în Command Window) în care scrieți colecții de comenzi pe mai multe linii, numite *scripturi*. Toate laboratoarele sunt organizate ca scripturi, iar voi sunteți încurajați să lucrați direct în Editor și să rulați întregul script (sau o secțiune) cu butonul **Run** sau tasta **F5**.

De fiecare dată când tastați o expresie în Command Window, Matlab o *evaluează* și afișează rezultatul. Aceasta se numește **mod interactiv**.

Câteva comenzi generale utile:

- `clc` — curăță Command Window;
- `clear` — șterge toate variabilele din Workspace;
- `doc numefunctie` sau `help numefunctie` — deschide pagina de documentație a unei funcții (de ex. `help sin`). Nu există nicio modalitate de a memora toate funcțiile Matlab; le descoperiți prin documentație.

2.2 Matematică de bază: Command Window ca un calculator

Tastați următoarele expresii, una câte una, și observați rezultatele:

```
2 + 2  
  
10 / 4  
  
2^10  
  
5 * (3 - 8)  
  
sqrt(2)
```

```
sin(pi / 2)
```

Matlab are operatorii aritmetici obișnuiți $+$, $-$, $*$, $/$ și $^$ (putere), și funcțiile obișnuite `sin`, `cos`, `tan`, `exp`, `log`, `sqrt`, `abs`, `round`, ... Parantezele $()$ se folosesc pentru grupare, exact ca în matematică.

Spre deosebire de un calculator obișnuit, Matlab cunoaște constantele speciale `pi`, `Inf` (infinit) și `NaN` (Not a Number — nu este un număr, de ex. rezultatul lui $0/0$). Încercați:

```
pi
1/0
0/0
```

Notă: convenția din Matlab folosește `log(x)` pentru logaritmul natural ($\ln x$). Logaritmul în bază 10 este `log10(x)`.

2.3 Variabile

O variabilă este un nume (începând cu o literă) care stochează o valoare. Atribuiți o valoare cu operatorul `=`:

```
a = 3
b = 2*a^2 + 5
name = "DEDP"
```

Valoarea variabilei `a` poate fi folosită în orice expresie ulterioară. Observați că după fiecare atribuire rezultatul este afișat în Command Window, iar noua variabilă apare în panoul Workspace.

Dacă puneți un punct și virgulă `;` la sfârșitul unei atribuirii, Matlab stochează în continuare variabila, dar **nu afișează** rezultatul:

```
a = 3;
b = 2*a^2 + 5;
```

Acesta este stilul standard în interiorul scripturilor: variabilele se creează silențios, iar noi afișăm doar ce vrem să vedem.

Inspectați-vă variabilele în panoul Workspace, sau cu comenzile `who` (listează doar numele) și `whos` (nume, dimensiuni și memorie):

```
whos
```

2.4 Funcții și cuvinte cheie

O **funcție** este o operație denumită care primește valori de intrare (argumente, în paranteze) și întoarce o ieșire. Matlab are sute de funcții încorporate pentru orice operație matematică: am întâlnit deja `sin`, `cos`, `sqrt`, `exp`, `log`, `abs`, `round`, `size`, `length`, `numel`, ...

O funcție poate avea mai multe argumente și mai multe ieșiri. Ieșirile se încapsulează în paranteze drepte:

```
% Un argument, o ieșire
x = sin(pi/2)

% Două argumente, o ieșire
m = mod(17, 5)      % restul împărțirii lui 17 la 5: 2

% Un argument, două ieșiri: valoarea maximă și poziția ei
v = [3, 9, 1, 7];
[mx, idx] = max(v)   % mx = 9, idx = 2
```

Încercați `help mod` și `help max` pentru a vedea cum arată pagina de ajutor încorporată. Pagina arată întotdeauna sintaxa de apelare a funcției, adică câte argumente acceptă și în ce ordine.

Un **cuvânt cheie** este un cuvânt care are un sens fix, încorporat în limbajul Matlab și care nu poate fi folosit ca nume de variabilă. Cele mai comune cuvinte cheie, care vor fi folosite constant în laboratoarele următoare, sunt:

- `if`, `else`, `elseif`, `end` — execuție condițională și sfârșitul unui bloc;
- `for`, `while` — bucle (execuție repetată);
- `function` — definiția unei funcții proprii;
- `return`, `break`, `continue` — controlul fluxului în interiorul funcțiilor și buclilor;
- `true`, `false` — valorile logice.

De exemplu, structura condițională are forma:

```
x = 3;
if x > 0
    disp('x is positive')
else
    disp('x is not positive')
end
```

Buclele repetă un bloc de comenzi. O buclă **for** se repetă de un număr fix de ori, iterând prin elementele unui vector:

```
% Suma numerelor de la 1 la 10
s = 0;
for k = 1:10
    s = s + k;
end
disp(s)      % 55
```

O buclă **while** se repetă cât timp o condiție este adevărată (condiția este reevaluată înainte de fiecare repetare):

```
% Prima putere a lui 2 care depășește 100
p = 1;
while p <= 100
    p = 2*p;
end
disp(p)      % 128
```

Observați că **for**, **while**, **if** și **function** delimitează toate blocul lor cu cuvântul cheie **end** (sau **end** și **else**, alternativ pentru **if**): aceasta este o regulă generală a limbajului Matlab.

O **funcție definită de utilizator** este o funcție pe care o scriem noi înșine, cu cuvântul cheie **function**. O funcție proprie se scrie de obicei într-un fișier propriu, cu același nume ca funcția (de ex. o funcție numită **myfunc** este stocată în fișierul **myfunc.m**). De exemplu, următorul fișier **mymean.m** definește o funcție care calculează media elementelor unui vector:

```
% File: mymean.m

function m = mymean(x)
% MYMEAN Mean of the elements of the vector x
    m = sum(x) / length(x);
end
```

După salvarea fișierului, funcția este apelată din Command Window sau din orice script, exact ca o funcție încorporată:

```
v = [1, 2, 3, 4, 5];  
m = mymean(v)      % 3
```

Prima linie a unei funcții se numește **semnătura**: declară argumentele de ieșire (aici **m**), numele funcției (**mymean**) și argumentele de intrare (aici **x**). Linia de comentariu de sub ea este așa-numita linie de ajutor H1, adică ceea ce afișează **help mymean** în Command Window. Intrările **x** și ieșirea **m** sunt variabile locale funcției: ele nu apar în Workspace și nu afectează restul scriptului.

Notă: funcția **disp()** afișează o valoare sau un text în Command Window, iar **%** începe un comentariu (tot ce urmează după **%** pe o linie este ignorat de Matlab). Le veți vedea pe ambele în orice script de acum înainte.

2.5 Vectori și matrici

Vectorii sunt cel mai important tip de date Matlab pentru acest curs: **un semnal este un vector**, iar **o imagine este o matrice**. Fiecare laborator va manipula vectori, deci această secțiune este fundația a tot ce urmează.

Crearea vectorilor și matricilor

Un **vector linie** se creează cu paranteze drepte **[]**, elementele fiind separate prin spații sau virgule:

```
v = [1, 2, 3, 4, 5]  
w = [1 2 3 4 5]
```

Un **vector coloană** separă elementele cu punct și virgulă:

```
v = [1; 2; 3; 4; 5]
```

O **matrice** combină cele două: liniile sunt separate prin punct și virgulă, iar elementele unei linii prin spații sau virgule:

```
M = [1, 2, 3; 4, 5, 6; 7, 8, 9]
```

Întrebare: care este diferența dintre vectorul linie **[1, 2, 3]** și vectorul coloană **[1; 2; 3]**?

Operatorul două puncte — șiruri regulat spațiate

Operatorul două puncte `a:step:b` creează un vector cu valorile $a, a + step, a + 2 \cdot step, \dots$ până la `b`. Valoarea din mijloc este **pasul**:

```
x = 0:1:10  
  
x = 0:0.1:1  
  
x = 10:-2:0
```

Observați că un pas negativ generează un șir descrescător. Dacă pasul este omis, el este 1: `0:10` este același lucru cu `0:1:10`. Ultima valoare a vectorului nu depășește niciodată `b` (pentru un pas pozitiv), deci numărul de elemente nu este neapărat fix.

Operatorul două puncte este modalitatea standard de a construi o **axă a timpului** pentru generarea semnalelor:

```
t = 0:0.01:1;      % o secundă de timp, eșantionată la fiecare 0,01 s (100 de eșantioane)  
x = sin(2*pi*5*t); % un semnal sinusoidal de 5 Hz, eșantionat pe axa t  
plot(t, x)
```

linspace — un număr dat de puncte între două valori

Funcția `linspace(a, b, n)` creează un vector cu exact n valori regulat spațiate între `a` și `b`, **inclusiv ambele capete**:

```
x = linspace(0, 1, 5)      % 5 puncte: 0, 0.25, 0.5, 0.75, 1  
  
x = linspace(0, 1, 100)    % 100 de puncte, pas 1/99
```

Diferența față de operatorul două puncte:

- `a:step:b` — fixăm **pasul** și lăsăm numărul de puncte să urmeze;
- `linspace(a, b, n)` — fixăm **numărul de puncte** și lăsăm pasul să urmeze (pasul este întotdeauna $(b - a)/(n - 1)$).
- `linspace(a, b, n)` conține întotdeauna capetele `a` și `b`, în timp ce operatorul două puncte poate sau nu să atingă exact `b`.

Ambele produc același tip de șir regulat și ambele sunt folosite pentru generarea axelor de timp și eșantionarea semnalelor, deci este bine să vă obișnuiți cu amândouă.

Dimensiuni și mărimi

Dimensiunea unui tablou (array) se obține cu `size()`, iar numărul total de elemente cu `numel()`:

```
M = [1, 2, 3; 4, 5, 6; 7, 8, 9];
size(M)

numel(M)

length(M)    % dimensiunea de-a lungul celei mai mari laturi
```

Un scalar este o matrice 1×1 : `size(5)` întoarce `1 1`.

2.6 Operații pe vectori și matrici

Matlab tratează fiecare variabilă ca pe o matrice. Aceasta dă două tipuri complet diferite de operații, iar *confundarea lor este cea mai frecventă greșeală a începătorilor*:

- **operații matriciale (algebră liniară)**: `*`, `/`, `^` — produsul matricial, inversa, puterea obișnuite;
- **operații element cu element**: `.*`, `./`, `.^` — aceeași operație aplicată separat fiecărui element (observați punctul!);

Pentru scalari (numere individuale) cele două sunt identice, motiv pentru care am putut scrie `2^10` mai sus. Pentru vectori și matrici ele sunt diferite:

```
a = [1, 2, 3];
b = [4, 5, 6];

a .* b          % produs element cu element: [4, 10, 18]

a .^ 2          % ridică fiecare element la pătrat: [1, 4, 9]

a * b          % EROARE: produsul matricial al unui 1x3 cu un 1x3 nu este definit
```

Formele element cu element sunt cele folosite pentru semnale: ele aplică o operație fiecărui eșantion al unui semnal.

2.7 Indexare: accesarea elementelor unui vector

Elementele unui vector se accesează cu paranteze (). Matlab indexează de la 1:

```
v = [10, 20, 30, 40, 50];  
v(1)      % primul element: 10  
  
v(3)      % al treilea element: 30  
  
v(2:4)    % elementele 2, 3, 4  
  
v(end)    % ultimul element
```

Pentru o matrice folosim doi indici, linia și coloana:

```
M = [1, 2, 3; 4, 5, 6];  
M(1, 2)    % linia 1, coloana 2: 2  
  
M(2, :)    % tot ce e pe linia 2: 4 5 6  
  
M(:, 3)    % tot ce e pe coloana 3: 3 6
```

Două punctele : singur înseamnă „toate elementele de-a lungul acelei dimensiuni”.

Indexare logică

Un vector poate fi indexat și cu un vector *logic* (un vector de valori **true/false**), care selectează doar elementele pentru care valoarea logică este **true**. Aceasta dă o modalitate foarte concisă de a filtra date:

```
v = [10, 3, 25, 8, 30, 1];  
sel = v > 8;      % vector logic: care elemente depășesc 8?  
v(sel)            % elementele însele: 10 25 30  
  
v(v > 8)          % același lucru, scris într-o singură linie
```

Indexarea logică este unealta pe care o vom folosi mai târziu pentru găsirea valorilor aberante (outliers), numărarea alarmelor false, extragerea de regiuni din imagini etc.

2.8 Grafică de bază

Funcția `plot()` desenează un vector ca o secvență de eșantioane, unite printr-o linie:

```
t = 0:0.01:1;
x = sin(2*pi*3*t);
plot(t, x)
```

Când o rulați, se deschide o fereastră de figură. Câteva lucruri de reținut:

- axa **x** a graficului **nu este automată**: ea este dată de primul argument (aici axa timpului **t**). `plot(x)` fără axă de timp ar reprezenta pur și simplu eșantioanele în funcție de indicii lor;
- figura poate fi adnotată cu `xlabel`, `ylabel`, `title` și `grid on`;
- `stem(t, x)` este un alt grafic util pentru semnale discrete, care desenează un marcaj pentru fiecare eșantion în loc de o linie. Folosiți `stem` când vreți să vedeți *eșantioanele* individuale ale unui semnal, și `plot` când vreți să vedeți *forma*.

```
t = 0:0.01:1;
x = sin(2*pi*3*t);
plot(t, x)
xlabel('Time (s)')
ylabel('Amplitude')
title('A 3 Hz sine wave')
grid on
```

2.9 Rezumat: funcții și notații pe care trebuie să le cunoașteți

Tabelul de mai jos adună tot ce a fost introdus în acest laborator. Veți folosi aceste funcții și această notație în fiecare laborator al acestui curs, deci păstrați-l ca referință.

Semnătură	Exemplu de apel	Descriere
Operatori		
<code>+</code> <code>-</code> <code>*</code> <code>/</code> <code>^</code>	<code>a + b</code> , <code>a * b</code> , <code>a ^ 2</code>	Operatori aritmetici de bază; Command Window funcționează ca un calculator

Semnătură	Exemplu de apel	Descriere
<code>.* ./ .^</code>	<code>u .* v, x .^ 2</code>	Operații element cu element: aplicate fiecărei perechi de elemente ale unor tablouri de aceeași dimensiune (operatorii matriciali fără punct funcționează diferit)
Mediul		
<code>clc</code>	<code>clc</code>	Curăță textul din Command Window
<code>clear</code>	<code>clear, clear x y</code>	Șterge toate (sau unele) variabile din Workspace
<code>help</code>	<code>help plot</code>	Afișează documentația unei funcții
<code>who, whos</code>	<code>whos</code>	Listează variabilele curente: who doar numele, whos adaugă dimensiunea și tipul
<code>disp</code>	<code>disp(x)</code>	Afișează valoarea unei expresii sau variabile
Funcții de bază		
<code>abs, sqrt</code>	<code>abs(-3), sqrt(2)</code>	Valoare absolută, rădăcină pătrată
<code>sin, cos, exp, log</code>	<code>sin(pi/2), log(10)</code>	Funcții trigonometrice și exponențiale; log este logaritmul natural (baza <i>e</i>), pentru baza 10 este nevoie de log10
<code>round</code>	<code>round(2.6)</code>	Rotunjește la cel mai apropiat întreg
<code>pi, Inf, NaN</code>	<code>2*pi, 1/0, 0/0</code>	Constante predefinite
<code>size, numel, length</code>	<code>size(M), numel(M), length(v)</code>	Dimensiunile unui tablou: size (<i>n</i> pe <i>m</i>), numărul de elemente, lungimea celei mai mari dimensiuni
<code>max</code>	<code>[mx, idx] = max(v)</code>	Valoarea maximă și (cu o a doua ieșire) poziția ei
Crearea și indexarea tablourilor		
<code>:</code> (două puncte)	<code>1:10, 0:0.1:1, 5:-1:1</code>	Creează un șir regulat spațiat a:step:b ; valoarea din mijloc este pasul, ultima valoare nu depășește niciodată b

Semnătură	Exemplu de apel	Descriere
[...]	[1 2 3], [1;2;3], [1 2; 3 4]	Creează un vector linie (spații sau virgule), un vector coloană (;) sau o matrice (; separă liniile)
linspace(a,b,n)	linspace(0,1,5)	Creează exact n puncte egal spațiate între a și b, inclusiv ambele capete
v(i), v(i:j)	v(3), v(2:4)	Indexare: elementul i, felia de la i la j
end	v(end), M(end,:)	Se referă la ultima poziție a unui tablou
M(i,:), M(:,j)	M(3,:), M(:,2)	Linia întreagă i / coloana întreagă j a unei matrice
indexare logică	v(v > 8), sel = v > 8; v(sel)	Selectează elementele pentru care o condiție este adevărată (true/false sunt 1/0 în aritmetică)
Grafică		
plot(t,x)	plot(t, x)	Desenează eșantioanele unite printr-o linie; primul argument este axa
stem(t,x)	stem(t, x)	Desenează fiecare eșantion ca un marcaj discret; de folosit pentru semnale discrete
xlabel, ylabel, title, grid	xlabel('Time (s)'), title('...'), grid on	Adnotări și grilă pentru figuri

3 Exerciții

Lucrați în Editor: scrieți un script (sau un live script `.mlx`) care conține toate comenzile de mai jos, rulați-l și observați rezultatele. Păstrați scriptul — este punctul de plecare al lucrării voastre individuale din acest curs.

1. Calculați, în Command Window:

- $7 \cdot 8 + 12$
- $(2^5 - 3^4) \cdot 6$
- $\frac{1}{1^2} + \frac{1}{2^2} + \frac{1}{3^2} + \dots + \frac{1}{10^2}$ (indiciu: calculați-l cu vectori și `sum()`, sau cu o buclă `for`)
- $\frac{4}{\pi^2}$ și comparați-l cu rezultatul de la punctul c. Ce recunoașteți?

2. Vectori și matrici:

- Creăți vectorul linie **a** = [3, 1, 4, 1, 5] și vectorul coloană **b** = [9; 2; 6; 5; 3].
- Creăți matricea

$$M = \begin{bmatrix} 1 & 2 & 3 & 4 \\ 5 & 6 & 7 & 8 \\ 9 & 10 & 11 & 12 \end{bmatrix}$$

- Folosind **size()**, afișați dimensiunile lui **a**, **b** și **M**.
 - Extrageți din **M** ultima coloană, primele două linii și elementul de pe linia 2, coloana 3.
3. Creăți vectorul **x** = 0:1:10 (11 elemente, de la 0 la 10).

- Afișați lungimea lui **x** cu **length()**.
- Calculați **sin(x*pi/4)** și rotunjiți valorile la numere întregi cu **round()**. Ce observați la valorile rotunjite?
- Calculați, într-o singură comandă, suma elementelor lui **x**, folosind funcția **sum()**. (Funcția **sum()** adună toate elementele unui vector — încercați să deduceți sensul ei ca nume pentru „totalul”.)

4. Semnale ca vectori:

- Creăți o axă a timpului **t** = 0:0.001:1 (1001 eșantioane).
- Generați semnalul **s** = **sin(2*pi*440*t)** și reprezentați-l grafic.
- Generați un al doilea semnal **s2** = **sin(2*pi*5*t)** (o sinusoidă de frecvență mai mică) și reprezentați **s** + **s2** în aceeași figură, cu **hold on** și două culori diferite. Cele două semnale se adună eșantion cu eșantion — încercați să explicați de ce funcționează.
- Repetăți punctul (a) cu pașii 0.1, 0.01 și 0.001 și observați cum se schimbă eșantionarea semnalului. Cum depinde numărul de eșantioane de pas?

5. Indexare logică:

- Creăți vectorul **v** = [1, 8, 3, 9, 5, 7].
- Calculați câte elemente ale lui **v** sunt mai mari decât 5, folosind **sum(v > 5)**, și un comentariu care explică trucul: valorile logice **true/false** sunt numărate ca 1/0 de **sum()**.
- Extrageți toate elementele lui **v** mai mari decât 5, folosind indexarea logică, și verificați că numărul lor corespunde răspunsului de la punctul b.

4 Întrebări finale

1. Ce face un punct și virgulă la sfârșitul unei comenzi? Când este util?
2. Care este diferența dintre `a*b` și `a.*b`? Dați un exemplu pentru fiecare.
3. Care este diferența dintre un vector linie și un vector coloană în Matlab, și cum se creează fiecare?
4. Cum accesați al treilea element al unui vector `v`? Cum accesați ultimul element al unui vector de lungime necunoscută?
5. Ce face comanda `whos`? Ce informații oferă?
6. Când este `stem()` de preferat față de `plot()`?